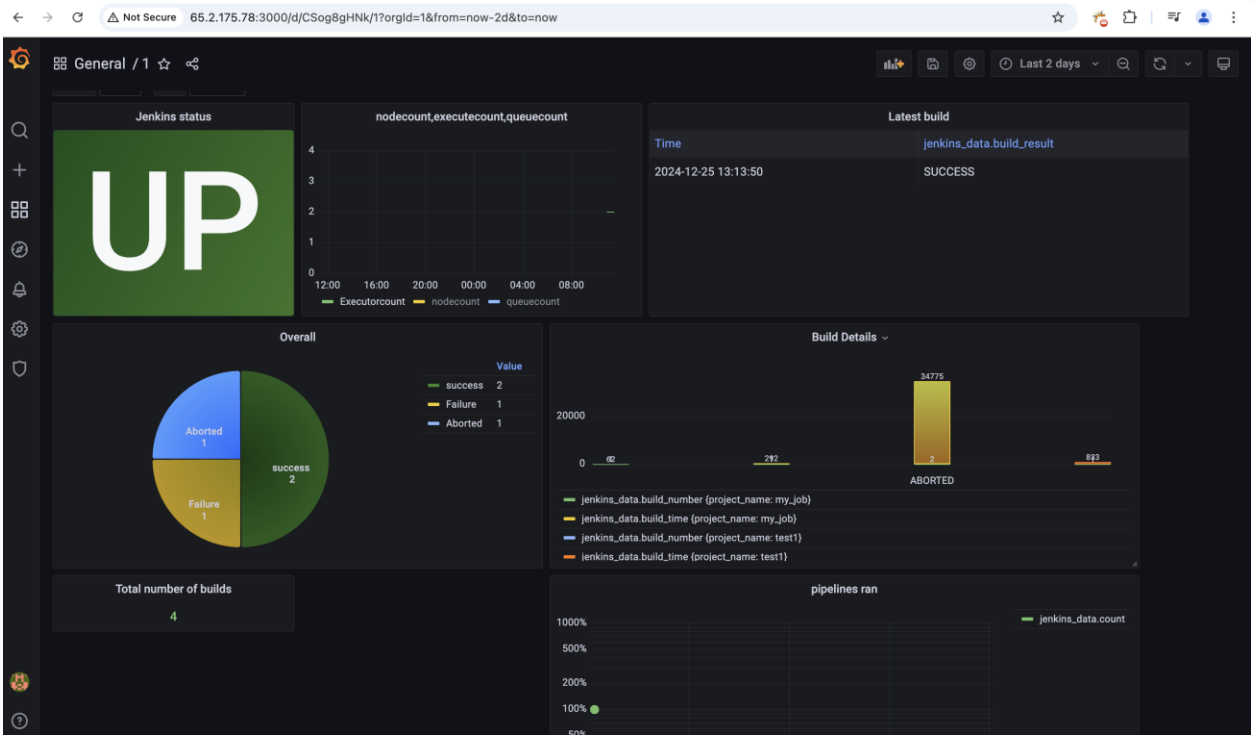
Production grade Jenkins using Prometheus, Grafana & InfluxDB



Monitoring Stack Setup Guide

Prerequisites

Ensure the following tools are installed and available:

- Docker
- Jenkins
- Prometheus
- Grafana
- InfluxDB
- SQL Queries

Setup Instructions

Install Required Plugins

1. **InfluxDB Plugin**
2. **Prometheus Plugin**

Grafana Setup

- Create a folder for Grafana.
- Configure jobs as needed.

Docker Installation on Ubuntu

Run the following commands to install Docker on an Ubuntu system:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh
```

Once Docker is installed successfully, proceed to the next steps.

Bring Up the Monitoring Stack Using Docker

Pull the Jenkins Image or Create It from a Dockerfile

To pull and run the Jenkins image, execute the following command:

```
docker run -d -p 8080:8080 jenkins/jenkins:lts-jdk17
```

Access the Jenkins Container

To access the Jenkins container, use the following command:

```
docker exec -it <container_id> /bin/bash
```

Replace <container_id> with the actual container ID of the Jenkins container. You can find the container ID by running:

```
docker ps
```

Retrieve the Initial Admin Password

Inside the Jenkins container, retrieve the initial admin password by running:

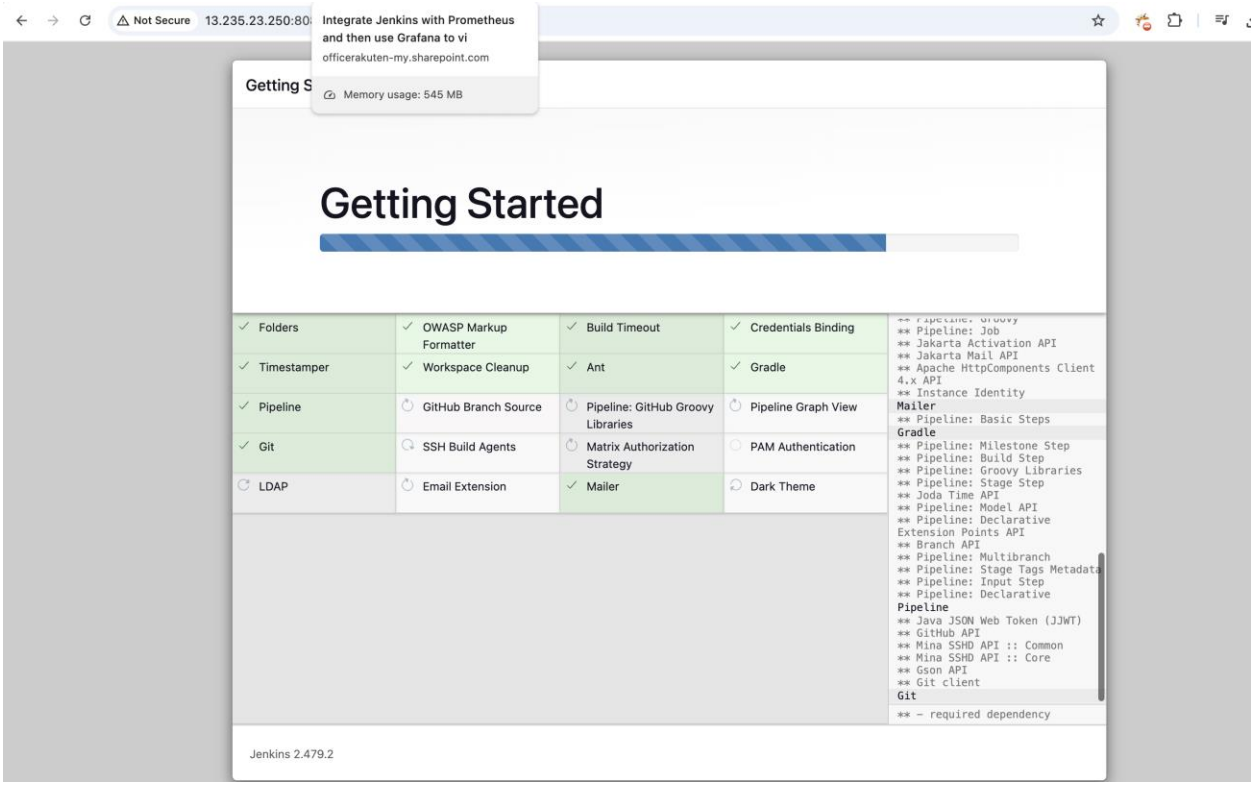
```
cat /var/jenkins_home/secrets/initialAdminPassword
```

Example output:

```
aff15ca7bac3491abc9b7345673bb1ac
```

Use this password to log in to the Jenkins web interface.

You can copy this content into a text editor or word processor and save it as a .docx or .pdf file. Let me know if you need further assistance!



Here's a well-structured and formatted version of your instructions for setting up **Prometheus** with Docker. You can copy this and save it as a document for your reference.

Prometheus Setup Guide

Prometheus Configuration

1. Navigate to the desired directory where you want to create the Prometheus configuration file:

cd /home/ubuntu

2. Create and edit the prometheus.yml file:

vi prometheus.yml

3. Add the following configuration to the prometheus.yml file:

```
global:
  scrape_interval: 15s      # How often to scrape targets by default.
  evaluation_interval: 15s   # How often to evaluate rules by default.
  scrape_timeout: 10s       # Timeout for scraping targets.

# Alerting rules configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:          # Alertmanager instances
          - "localhost:9093"

# Rule files for recording and alerting rules
rule_files:
  # List of files containing alerting and recording rules
  - "alert.rules"

# Scrape configuration
scrape_configs:
  # Scrape configuration for Prometheus itself
  - job_name: "jenkins"
    static_configs:
      - targets:
          - "localhost:8080"
        metrics_path: /prometheus
```

Running Prometheus with Docker

1. Use the following command to run Prometheus in a Docker container:

```
docker run -d --name prometheus-container \
  -v /home/ubuntu/prometheus.yml:/etc/prometheus/prometheus.yml \
  -e TZ=UTC \
  -p 9090:9090 \
```

ubuntu/prometheus:2.33-22.04_beta

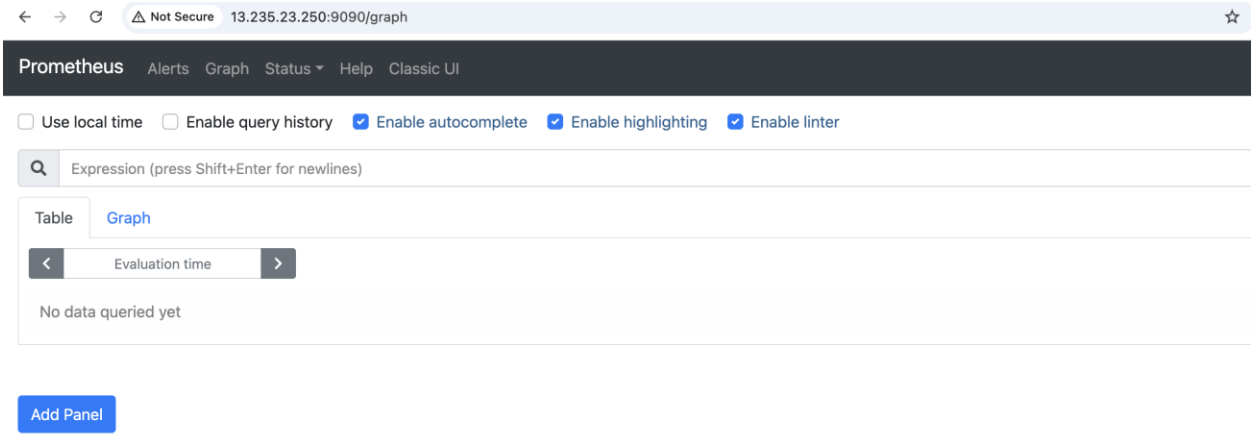
a. -d: Run the container in detached mode.
b. --name prometheus-container: Assign a name to the container.
c. -v /home/ubuntu/prometheus.yml:/etc/prometheus/prometheus.yml: Mount the Prometheus configuration file from the host to the container.
d. -e TZ=UTC: Set the timezone to UTC.
e. -p 9090:9090: Map port 9090 of the container to port 9090 on the host.
f. ubuntu/prometheus:2.33-22.04_beta: Use the specified Prometheus image.

2. Once the container is running, you can access the Prometheus web interface by navigating to:

http://<your-server-ip>:9090

Replace <your-server-ip> with the IP address of your server.

This document provides a clear and concise guide for setting up Prometheus with Docker. You can save this as a .docx or .pdf file for future use. Let me know if you need further assistance!



Grafana

Here's a detailed guide and content for setting up **Grafana** using Docker. You can use this as part of your documentation or guide.

Grafana Setup Guide

What is Grafana?

Grafana is an open-source platform for monitoring, visualization, and alerting. It allows you to query, visualize, and analyze metrics from various data sources such as Prometheus, InfluxDB, Elasticsearch, and more. Grafana provides customizable dashboards and supports alerting to help you monitor your systems effectively.

Running Grafana with Docker

To set up Grafana using Docker, follow these steps:

Step 1: Pull the Grafana Docker Image

Before running Grafana, ensure you have Docker installed on your system. Then, pull the Grafana Docker image:

```
docker pull grafana/grafana:8.5.5
```

This command pulls the specific version 8.5.5 of the Grafana image from Docker Hub.

Step 2: Run the Grafana Container

Run the following command to start a Grafana container:

```
docker run -d --name=grafana -p 3000:3000 grafana/grafana:8.5.5
```

Explanation of the Command:

- -d: Runs the container in detached mode (in the background).
- --name=grafana: Assigns the name grafana to the container.
- -p 3000:3000: Maps port 3000 of the container to port 3000 on the host machine. Grafana's web interface is accessible on this port.
- grafana/grafana:8.5.5: Specifies the Grafana image and version to use.

Step 3: Access the Grafana Web Interface

1. Once the container is running, open your web browser.
2. Navigate to the following URL:

http://<your-server-ip>:3000

Replace <your-server-ip> with the IP address of your server or localhost if running locally.

3. You will see the Grafana login page. Use the default credentials to log in:
    a. **Username**: admin

   b. **Password**: admin
4. After logging in for the first time, you will be prompted to change the default password.

Step 4: Configure Data Sources

1. After logging in, navigate to **Configuration > Data Sources**.
2. Add a data source (e.g., Prometheus, InfluxDB, etc.) by selecting the appropriate option and providing the required connection details.
3. Save and test the data source to ensure it is working correctly.

Step 5: Create Dashboards

1. Navigate to **Dashboards > New Dashboard**.
2. Add panels to visualize metrics from your configured data sources.
3. Customize the panels and layout as needed.

docker rm -f grafana

Additional Notes

- **Persistent Storage**: By default, Grafana stores its data inside the container. To persist data (e.g., dashboards, configurations), you can mount a volume to the container:

```
docker run -d --name=grafana -p 3000:3000 \
  -v /path/to/grafana/data:/var/lib/grafana \
  grafana/grafana:8.5.5
```

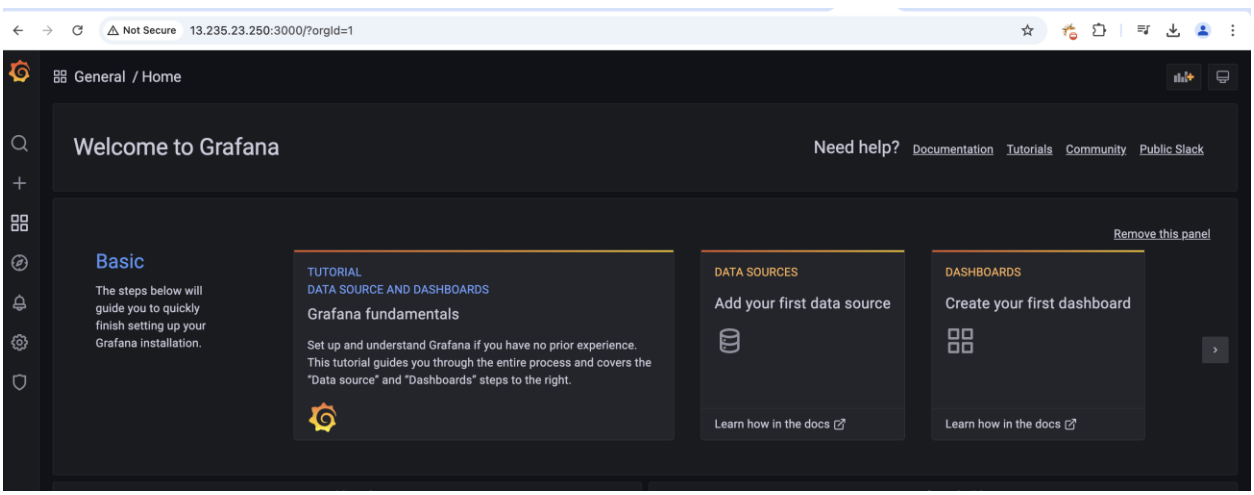
Replace /path/to/grafana/data with the directory on your host where you want to store Grafana data.

- **Custom Configuration**: You can provide a custom Grafana configuration file by mounting it to the container:

```
docker run -d --name=grafana -p 3000:3000 \
 -v /path/to/custom/grafana.ini:/etc/grafana/grafana.ini \
 grafana/grafana:8.5.5
```

- **Upgrading Grafana**: To upgrade Grafana, stop and remove the existing container, pull the latest image, and start a new container with the updated image.

This guide provides a comprehensive overview of setting up and running Grafana using Docker. Let me know if you need further assistance or additional details!



Influx DB :

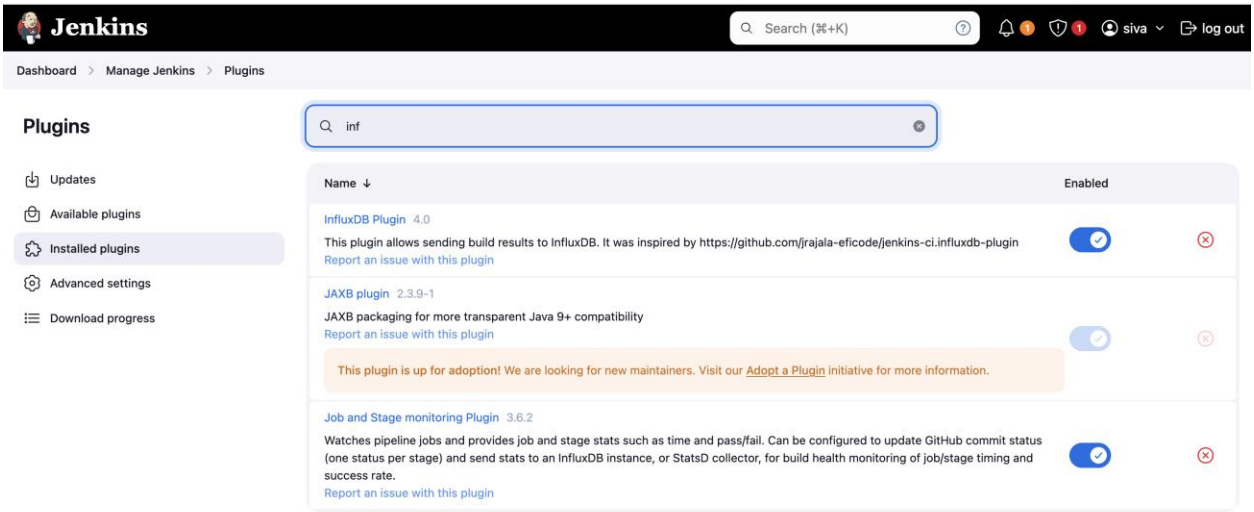
docker run -d -p 8086:8086 --name influxdb2 influxdb:1.8.6-alpine

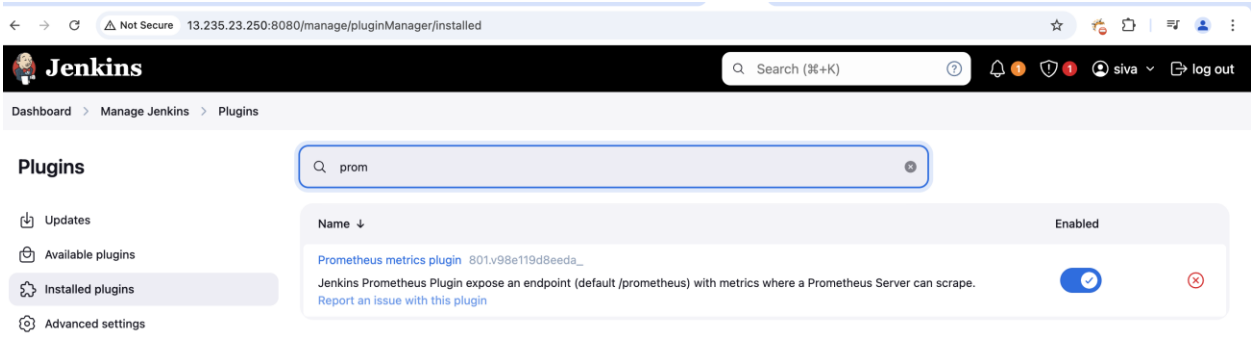
```
root@ip-172-31-36-244:/home/ubuntu# docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED              STATUS              PORTS
                                                   NAMES
0d881d77f040   influxdb:1.8.6-alpine                  "/entrypoint.sh infl…"   About a minute ago   Up About a minute   0.0.0.0:
8086->8086/tcp, :::8086->8086/tcp                  influxdb2
d02909acc270   grafana/grafana:8.5.5                  "/run.sh"                7 minutes ago        Up 7 minutes        0.0.0.0:
3000->3000/tcp, :::3000->3000/tcp                  grafana
1351613c9bef   ubuntu/prometheus:2.33-22.04_beta      "/usr/bin/prometheus…"   21 minutes ago       Up 21 minutes       0.0.0.0:
9090->9090/tcp, :::9090->9090/tcp                  prometheus-container
6911c84f312a   jenkins/jenkins:lts-jdk17              "/usr/bin/tini -- /u…"   32 minutes ago       Up 32 minutes       0.0.0.0:
8080->8080/tcp, :::8080->8080/tcp, 50000/tcp       intelligent_gates
```

Connect to container and execute below Influx Commands

docker exec -it influxdb2 bash
influx
CREATE DATABASE "jenkins" WITH DURATION 1825d REPLICATION 1 NAME "jenkins-retention"
SHOW DATABASES
USE jenkins

Install plugins in jenkins





http://13.235.23.250:8080/prometheus/

Not Secure 13.235.23.250:8080/manage/configure

Dashboard > Manage Jenkins > System >

InfluxDB Targets

Description ?

influxdb

URL ?

http://13.235.23.250:8086

Credentials

- none -

+ Add

Organization ?

Database / Bucket ?

jenkins

Retention Policy ?

jenkins-retention

Connection success

Test Connection

jenkins-retention

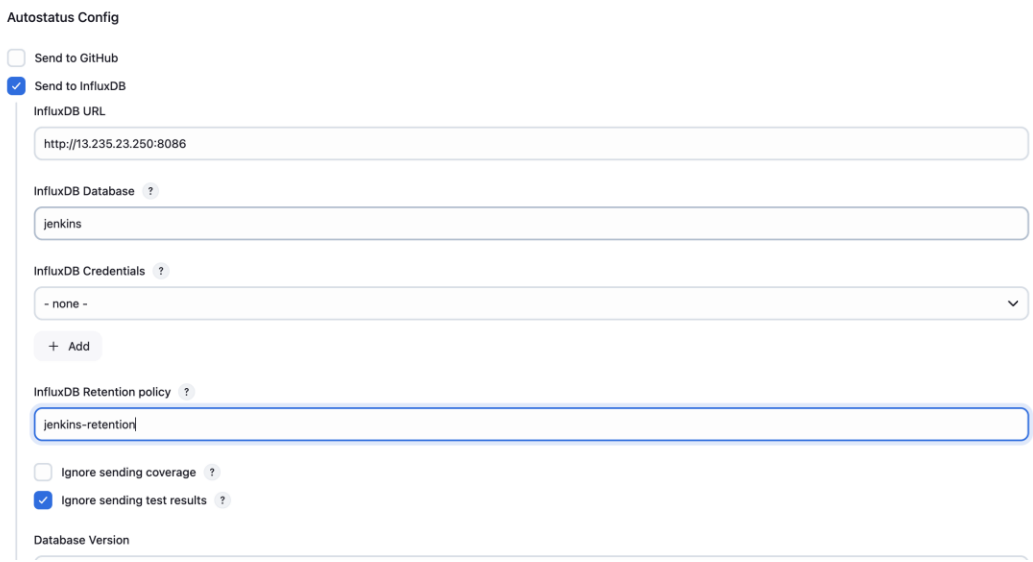
Connection success

Test Connection

- [x] Job scheduled time as timestamp ?
- [x] Expose Exceptions ?
- [ ] Use Jenkins Proxy ?
- [x] Global Listener ?

Global Listener Filter ?

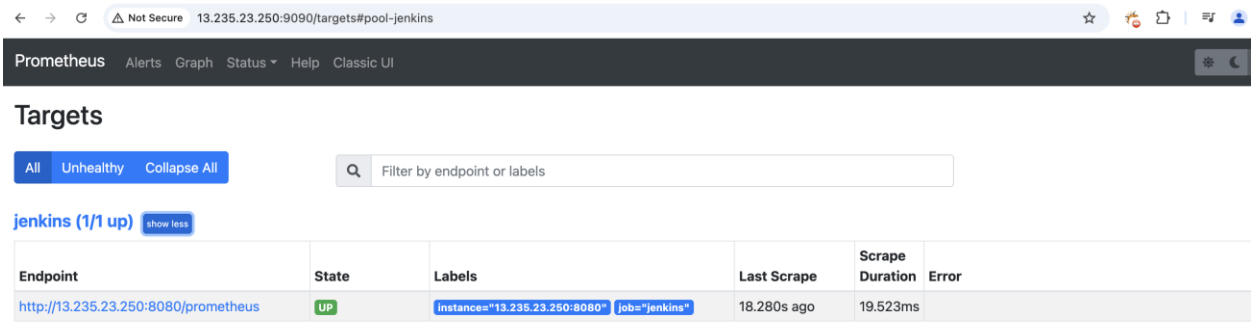
Add



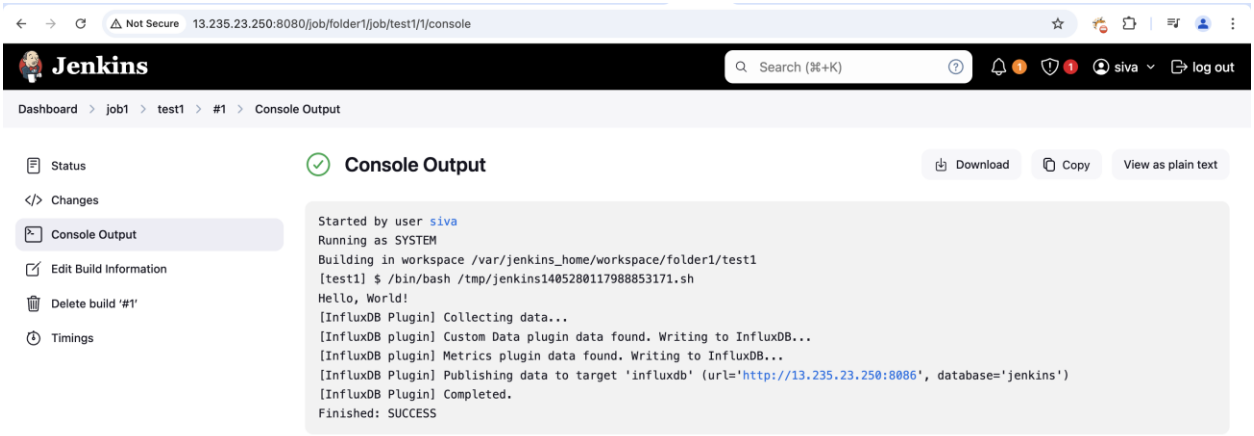
Vi promethus.yaml

```
# Example scrape configuration for custom applications
  - job_name: "jenkins"
    metric_path: '/prometheus'
    static_configs:
      - targets:
          - "13.235.23.250:8080"
```

Docker restart promethus containeriD



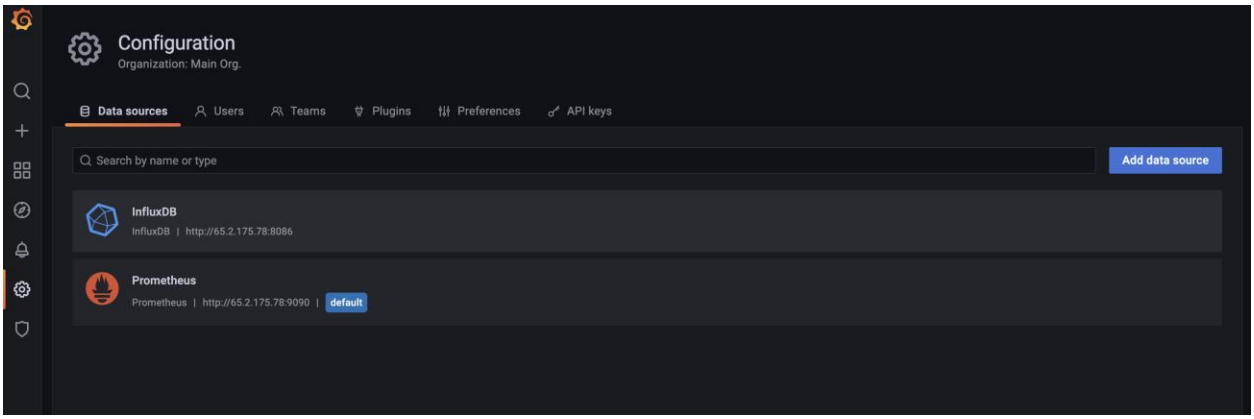
Basic job . Iam able to see influxdb (it will show in all logs)

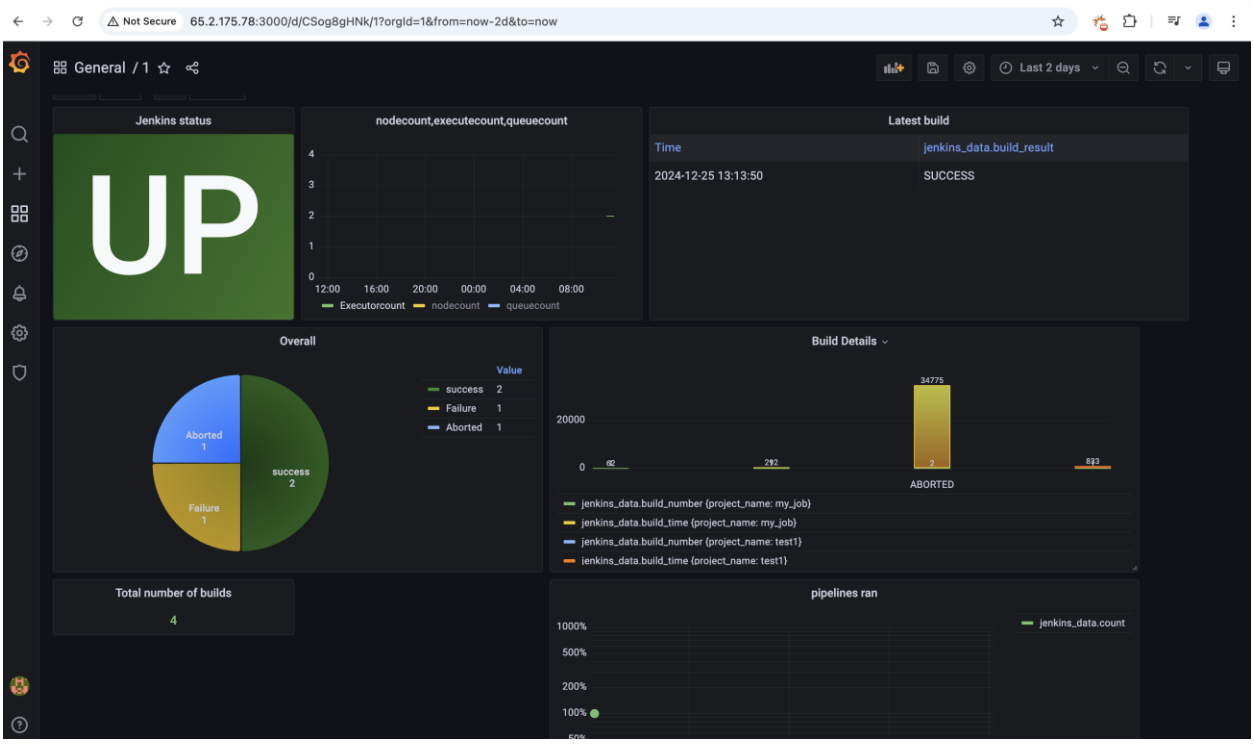


To check influxDB

```
root@ip-172-31-36-244:/home/ubuntu# docker exec -it 0d881d77f040 /bin/bash
bash-5.0# influx
Connected to http://localhost:8086 version 1.8.6
InfluxDB shell version: 1.8.6
> use jenkins
Using database jenkins
> SHOW MEASUREMENTS
name: measurements
name
----
agent_data
jenkins_data
job
metrics_data
```

Grafana;





Queries in grafana :

Jenkins status

```
up{instance="65.2.175.78:8080", job="jenkins"}
```

nodecount,executecount,queuecount

```
jenkins_executor_count_value{instance="65.2.175.78:8080", job="jenkins"}
jenkins_node_count_value{instance="65.2.175.78:8080", job="jenkins"}
jenkins_queue_blocked_value{instance="65.2.175.78:8080", job="jenkins"}
```

Latest build status:

SELECT "build_result"

FROM "jenkins_data"

ORDER BY time DESC

LIMIT 1

1. **Dropdown**
   a. SHOW TAG VALUES FROM job WITH KEY = "owner"
   b. SHOW TAG VALUES FROM job WITH KEY = repo WHERE "owner" =~ /^(f

2. **Overall Panel**
   a. Query to get success build count
   b. Query to get failed build count
   c. Query to get aborted build count
   d. Query to get unstable build count
   e. Number of pipelines ran
   f. Total number of builds
   g. Avg Build time
   h. Latest Build Status
   i. Build details